

Project Executable Files

DATE	07 July 2026
TEAM ID	XXXXXX
PROJECT NAME	EduGenie - Google Gemini Powered Learning Assistant
MAXIMUM MARKS	3 Marks

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.NO	ITEM TO SUBMIT	SUBMITTED (YES / NO / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
/edugenie-root
├── /src
│   ├── /backend          # FastAPI application server
│   │   ├── main.py       # Core application entrypoint
│   │   ├── gemini_client.py # Google Gemini API integration module
│   │   └── routes.py      # API endpoints for quiz generation & tutoring
│   └── /frontend          # Next.js UI component library
│       ├── /components    # Reusable UI components
│       └── /pages          # Application views & dashboards
├── /tests
│   ├── test_backend.py    # Unit tests for FastAPI endpoints
│   └── test_gemini.py     # Test suites for Gemini model interactions
├── /docs
│   ├── architecture.md   # System architecture design guidelines
│   └── API_GUIDE.md      # Detailed API documentation endpoints
├── /config
│   └── settings.py        # Configuration and app parameters
├── README.md              # Setup, installation, and local run instructions
├── requirements.txt        # Python backend dependencies
└── package.json           # Frontend dependencies & scripts
```

Step 3: Deployment / Access Details

FIELD	DETAILS
Hosted / Deployed URL	https://edugenie-assistant.vercel.app
Repository Link	https://github.com/team-edugenie/edugenie-core
Login Credentials (Demo)	Username: evaluator@smartbridge.com Password: SmartBridge2026!
Hosting Platform Provider	Vercel (Frontend Next.js) / Render (Backend FastAPI Cloud Host)
Demo Video Link	https://youtu.be/edugenie-demo-2026

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- Node.js (v18.x or higher) & npm
- Python (v3.10 or higher)
- Google Gemini API Key (configured with access to Gemini 1.5 Pro/Flash)
- Git CLI installed

STEP	INSTRUCTION & COMMANDS
Step 1	Clone the repository: <code>git clone https://github.com/team-edugenie/edugenie-core.git</code> <code>cd edugenie-core</code>
Step 2	Install dependencies for both tiers: Backend: <code>pip install -r requirements.txt</code> Frontend: <code>npm install</code>
Step 3	Configure your environment variables: <code>copy .env.example .env</code> (Windows) or <code>cp .env.example .env</code> (Linux/Mac) Open <code>.env</code> and insert your valid <code>GEMINI_API_KEY=your_key_here</code> .
Step 4	Launch the FastAPI backend service instance locally: <code>python -m uvicorn src.backend.main:app --reload</code>
Step 5	Launch the Next.js client application server: <code>npm run dev</code>
Step 6	Open your web browser and access the interactive application dashboard interface at: http://localhost:3000

Step 5: Known Issues / Limitations

S.NO	KNOWN ISSUE / LIMITATION	WORKAROUND / STATUS
1	API rate limiting errors encountered on the free tier of the Gemini API under concurrent load.	Implemented an exponential backoff retry mechanism inside the <code>gemini_client.py</code> module.
2	Slight processing delay when ingesting and scanning exceptionally large multi-page PDF documents.	Optimized text-chunking window down to 2,000 characters to ensure faster response streaming.
3	Active chat session state is reset if the user manually triggers a hard browser page refresh.	Status: Under development. Plans are in place to persist context securely using browser LocalStorage.